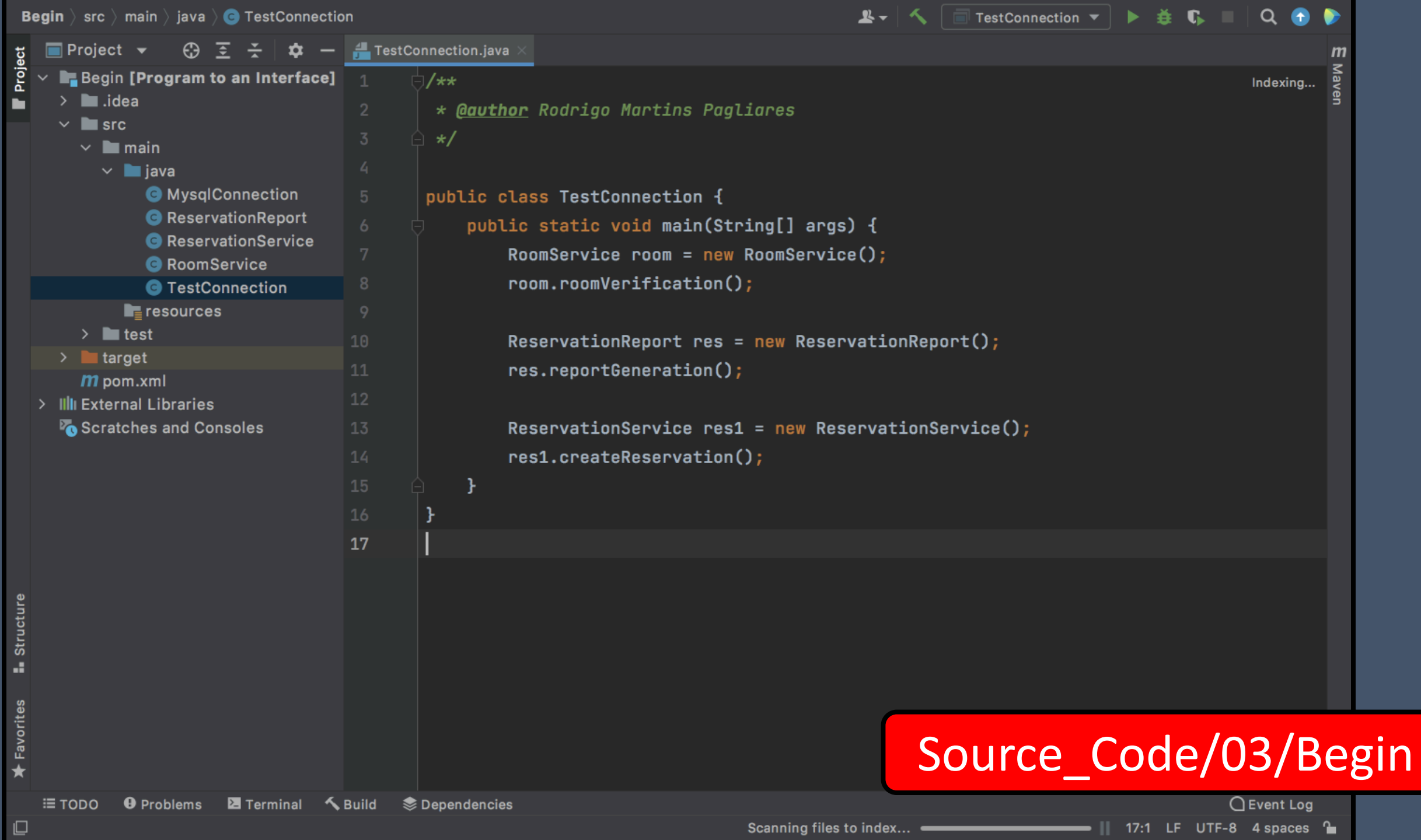


Design Patterns

Rodrigo Martins Pagliares

Design Patterns

Design principle:
Program to an interface and
not to an implementation



Source_Code/03/Begin

Project

Begin [Program to an Interface]

- .idea
- src
 - main
 - java
 - MySQLConnection
 - ReservationReport
 - ReservationService
 - RoomService
 - TestConnection**
 - resources
 - test
 - target
 - pom.xml
- External Libraries

```
5 public class TestConnection {
6     public static void main(String[] args) {
7         RoomService room = new RoomService();
8         room.roomVerification();
9
10        ReservationReport res = new ReservationReport();
11        res.reportGeneration();
12
13        ReservationService res1 = new ReservationService();
14        res1.createReservation();
15    }
16 }
```

Maven

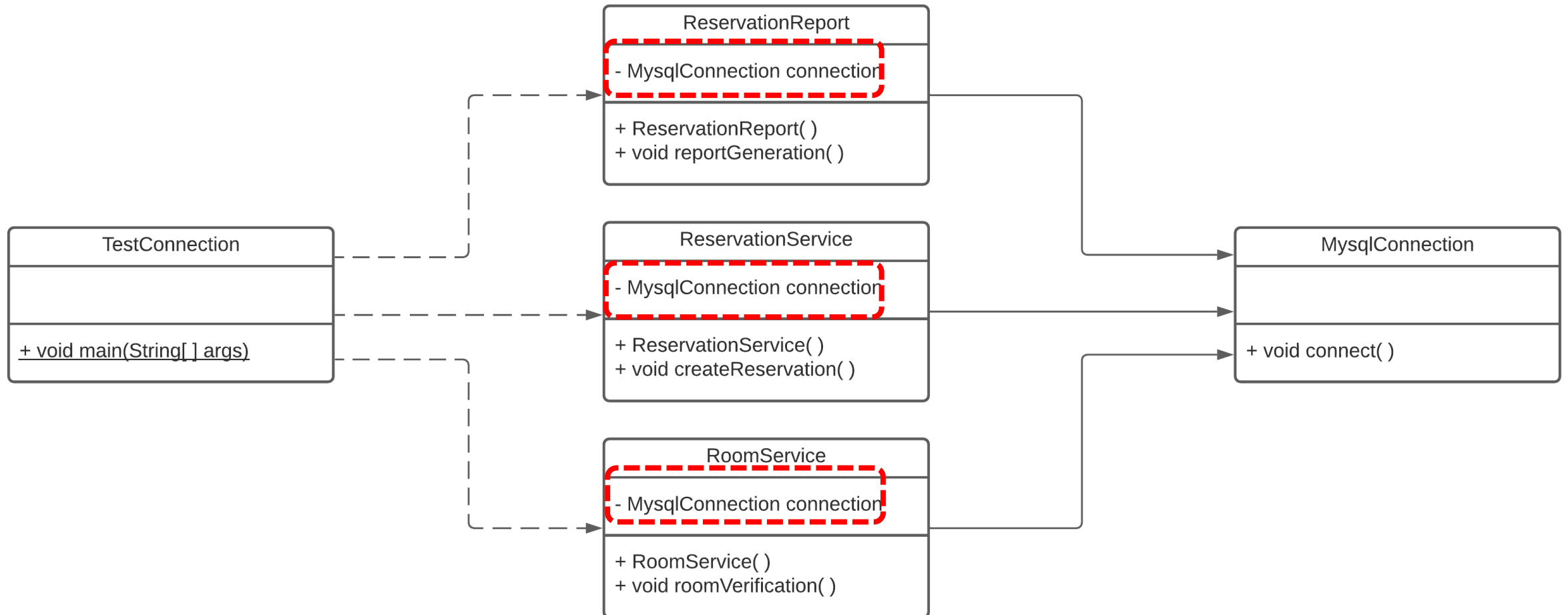
Run: TestConnection

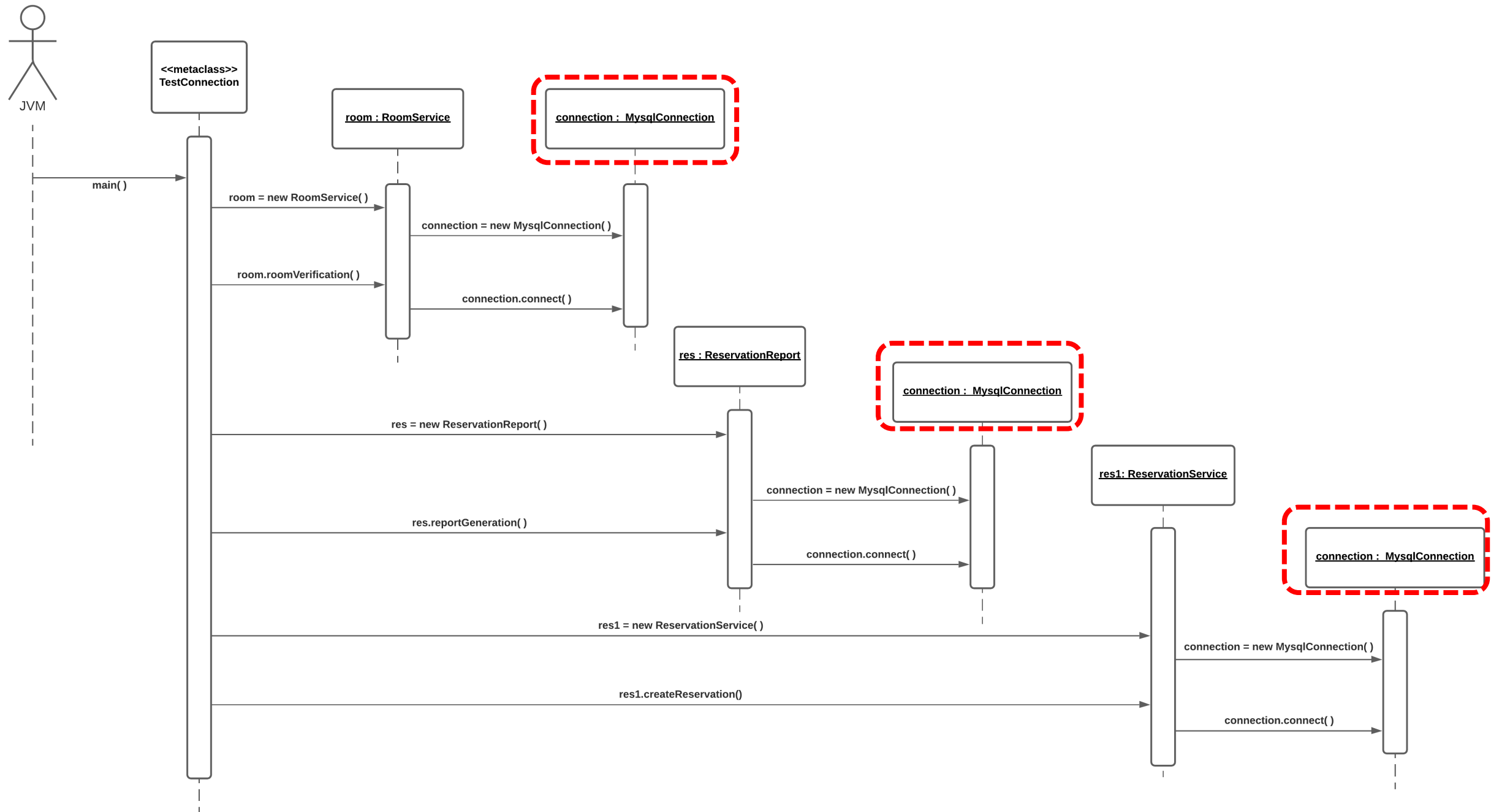
```
/Library/Java/JavaVirtualMachines/adoptopenjdk-16.jdk/Contents/Home/bin/java -javaagent:/Applications/IntelliJ IDEA CE
Connecting to MySQL
business logic over entity room
Connecting to MySQL
business logic in order to generate a report
Connecting to MySQL
business logic over the creation of a reservation

Process finished with exit code 0
```

Program output.

Class diagram: version #1





**Version 2: Suppose you want
to add support to Oracle**

Intermediate > src > main > java > OracleConnection

Project

- Intermediate ~ /Dropbox/Disciplina
 - .idea
 - src
 - main
 - java
 - MySQLConnection
 - OracleConnection
 - ReservationReport
 - ReservationService
 - RoomService
 - TestConnection
 - resources
 - test
 - target
 - pom.xml
 - External Libraries
 - Scratches and Consoles

OracleConnection.java

```
1  /**
2      * @author Rodrigo Martins Pagliares
3      */
4
5  public class OracleConnection {
6
7      public void connect() {
8          System.out.println("Connecting to Oracle");
9      }
10 }
11
```

Run: TestConnection

Structure

Run

Build completed successfully in 11 sec, 672 ms (moments ago)

Event Log

1:1 LF UTF-8 4 spaces

First we need to create the OracleConnection class that implements the Connection interface

OracleConnection.java x ReservationReport.java x

```
1  /**
2   * @author Rodrigo Martins Pagliares
3   */
4
5  public class ReservationReport {
6      private OracleConnection connection;
7
8      public ReservationReport() {
9          connection = new OracleConnection();
10     }
11
12     public void reportGeneration() {
13         connection.connect();
14         System.out.println("business logic in order to generate a report");
15     }
16 }
17
```

The problem is that we need to change 3 classes (ReservationReport, ReservationService, and RoomService) in order to replace the MySqlConnection with OracleConnection.

```
Project
OracleConnection.java x ReservationReport.java x ReservationService.java x
1 /**
2  * @author Rodrigo Martins Pagliares
3  */
4
5 public class ReservationService {
6     private OracleConnection connection;
7
8     public ReservationService() {
9         connection = new OracleConnection();
10    }
11
12    public void createReservation() {
13        connection.connect();
14        System.out.println("business logic over the creation of a reservation");
15    }
16
17
18
19 }
20
Structure
Favorites
```

Change from MySqlConnection to OracleConnection.

Project OracleConnection.java x ReservationReport.java x ReservationService.java x RoomService.java x

Maven

```
1  /**
2   * @author Rodrigo Martins Pagliares
3   */
4
5  public class RoomService {
6      private OracleConnection connection;
7
8      public RoomService() {
9          connection = new OracleConnection();
10     }
11
12     public void roomVerification() {
13         connection.connect();
14         System.out.println("business logic over entity room");
15     }
16 }
17
```

Change from MySqlConnection to OracleConnection.

Intermediate > src > main > java > TestConnection

Project

- Intermediate ~ /Dropbox/Disciplina
 - .idea
 - src
 - main
 - java
 - MySQLConnection
 - OracleConnection
 - ReservationReport
 - ReservationService
 - RoomService
 - TestConnection**
 - resources
 - test
 - target
 - pom.xml
 - External Libraries
 - Scratches and Consoles

Run: TestConnection

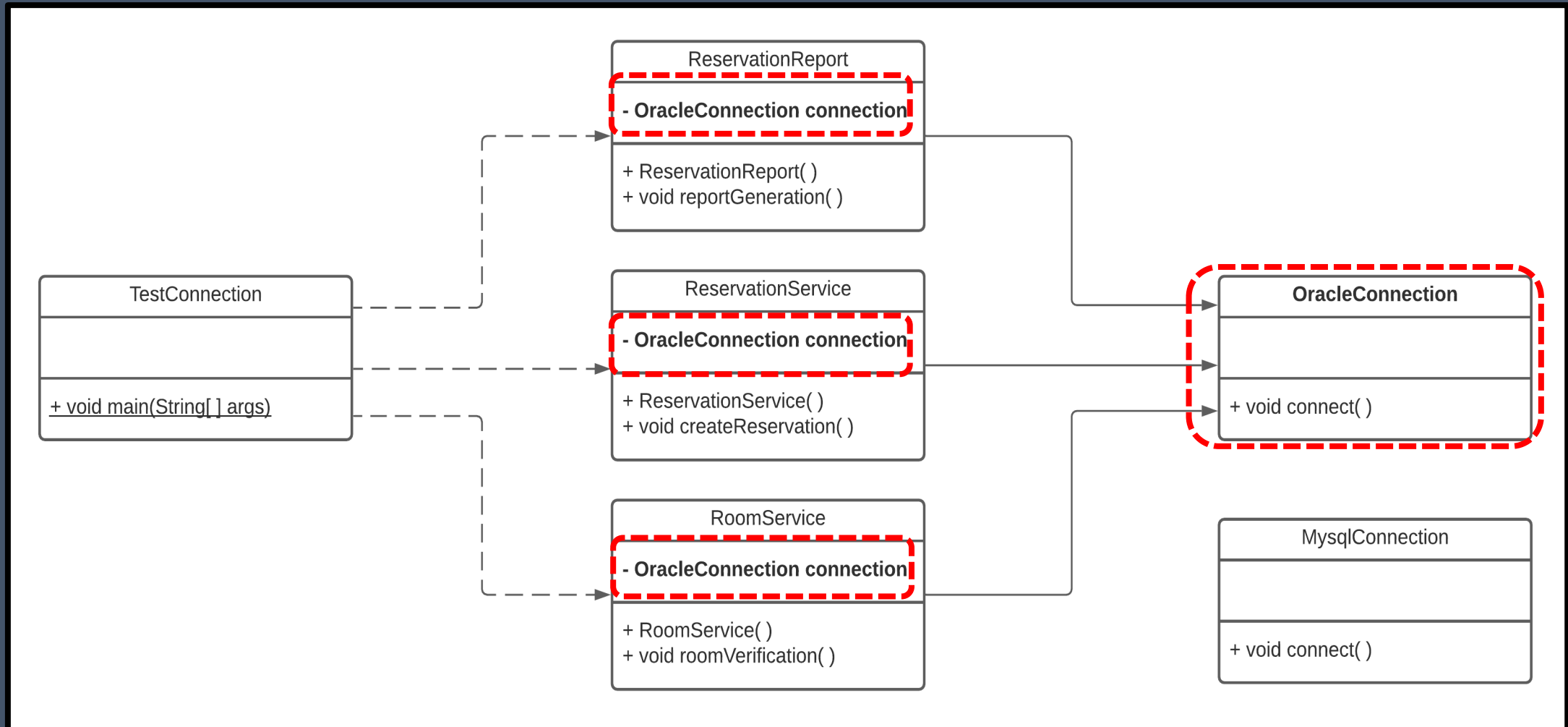
```
4  
5 public class TestConnection {  
6     public static void main(String[] args) {  
7         RoomService room = new RoomService();  
8         room.roomVerification();  
9  
10        ReservationReport res = new ReservationReport();  
11        res.reportGeneration();  
12  
13        ReservationService res1 = new ReservationService();  
14        res1.createReservation();  
15    }  
16 }  
17
```

Connecting to Oracle
business logic over entity room
Connecting to Oracle
business logic in order to generate a report
Connecting to Oracle
business logic over the creation of a reservation
Process finished with exit code 0

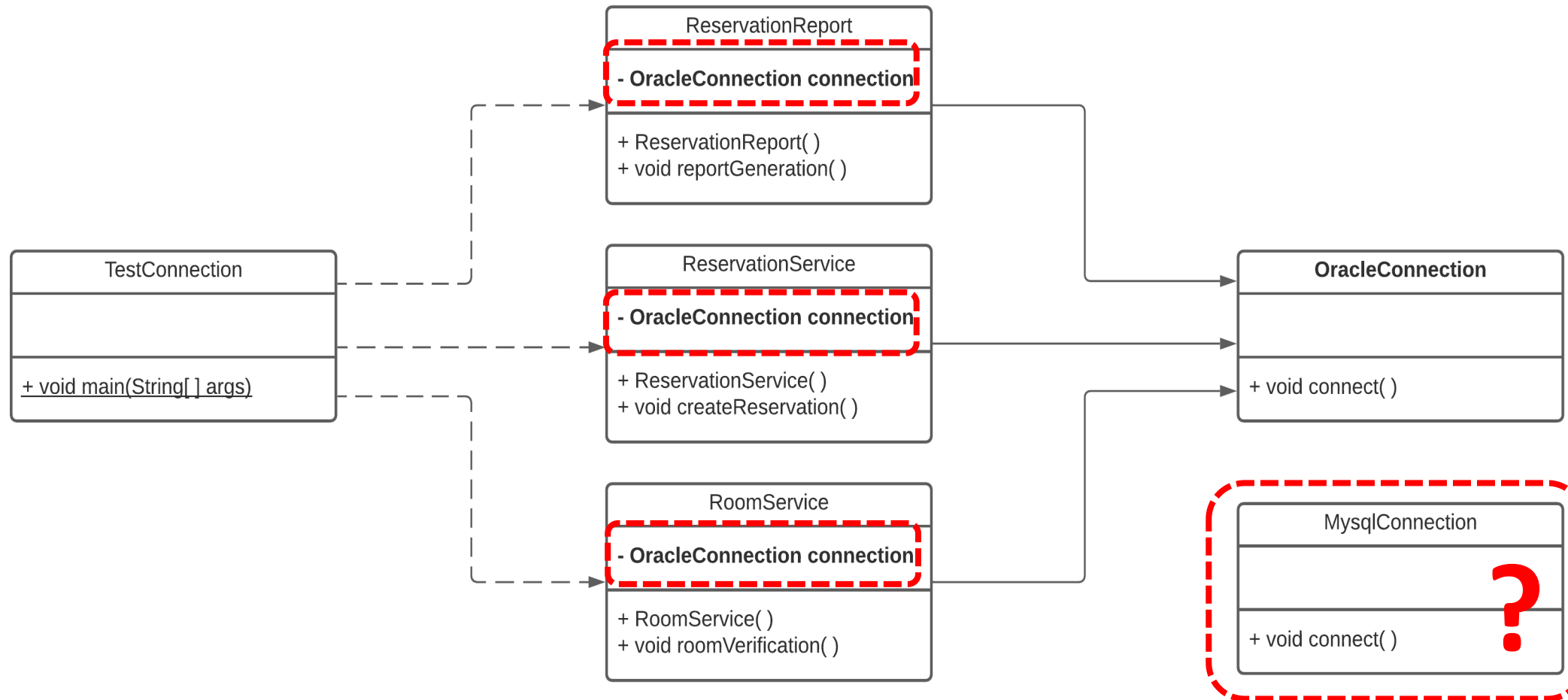
Build completed successfully in 2 sec, 936 ms (moments ago)

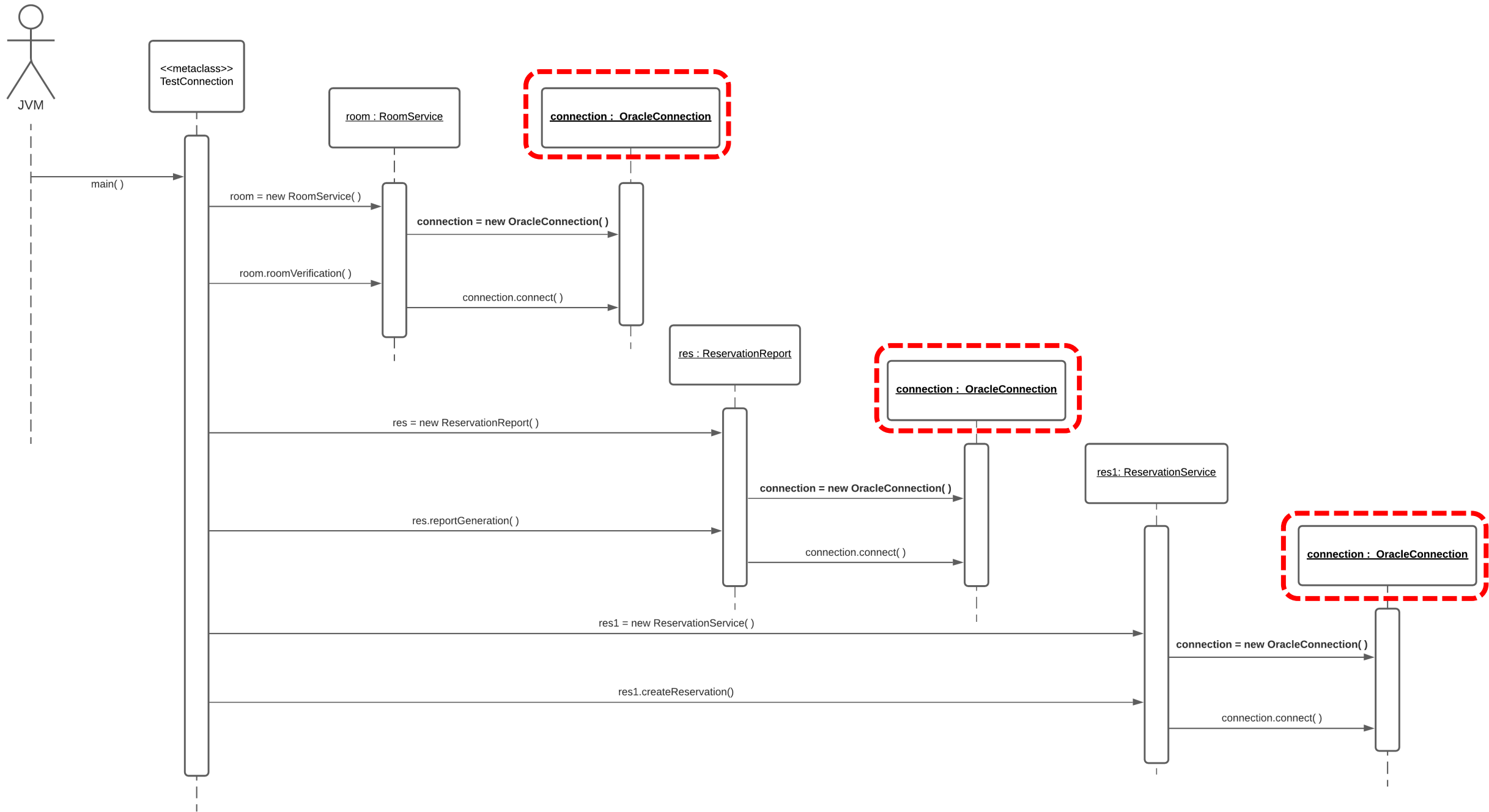
Program output. Now it works with Oracle instead of MySQL.

Class diagram: version #2

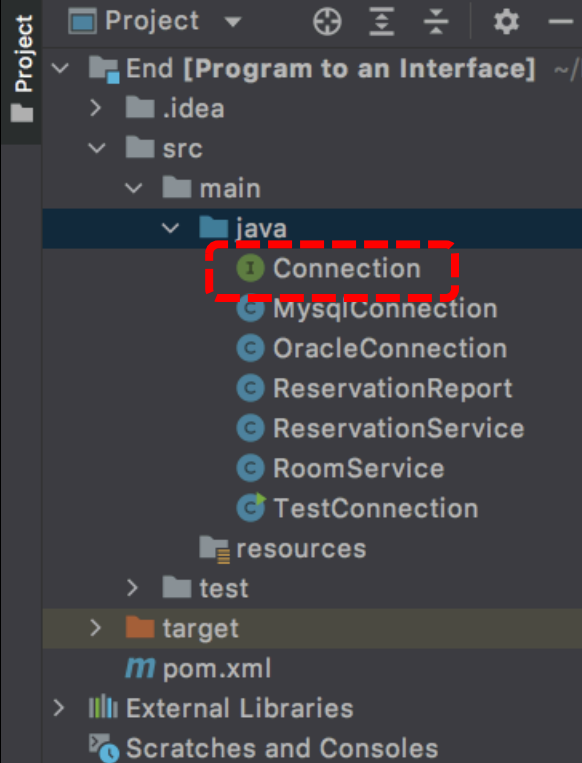


It now works with **Oracle**, but there is **no way** to work with **MySQL** without changing back the **ReservationReport**, **ReservationService** and **RoomService** classes.





**Version 3: Suppose you want
to support more than one
RDBMS**



```
1 /**
2  * @author Rodrigo Martins Pagliares
3  */
4
5 public interface Connection {
6     public void connect();
7 }
8
```

Create an interface named Connection

Run: TestConnection

End > src > main > java > MySqlConnection

Project

- End [Program to an Interface] ~/l
 - .idea
 - src
 - main
 - java
 - Connection
 - MySqlConnection**
 - OracleConnection
 - ReservationReport
 - ReservationService
 - RoomService
 - TestConnection
 - resources
 - test
 - target
 - pom.xml
 - External Libraries
 - Scratches and Consoles

TestConnection.java x Connection.java x MySqlConnection.java x

```
1  /**
2      * @author Rodrigo Martins Pagliares
3      */
4
5  public class MySqlConnection implements Connection {
6
7      @Override
8      public void connect() {
9          System.out.println("Connecting to MySQL");
10     }
11 }
12
```

Run: TestConnection x

Run | TODO | Problems | Terminal | Build | Dependencies | Event Log

Build completed successfully in 10 sec, 906 ms (6 minutes ago)

12:1 LF UTF-8 4 spaces

Update the MySqlConnection class to Implement the interface Connection. The @Override annotation is optional, but it is a good practice to use it to indicate we are overriding the interface method.

End > src > main > java > OracleConnection > connect

Project

- End [Program to an Interface] ~/l
 - .idea
 - src
 - main
 - java
 - Connection
 - MysqlConnection
 - OracleConnection
 - ReservationReport
 - ReservationService
 - RoomService
 - TestConnection
 - resources
 - test
 - target
 - pom.xml
 - External Libraries
 - Scratches and Consoles

TestConnection.java x Connection.java x MysqlConnection.java x OracleConnection.java x

```
1  /**
2    * @author Rodrigo Martins Pagliares
3    */
4
5  public class OracleConnection implements Connection {
6
7      @Override
8      public void connect() {
9          System.out.println("Connecting to Oracle");
10     }
11 }
12
```

Run: TestConnection x

Structure

Run | TODO | Problems | Terminal | Build | Dependencies

Event Log

Build completed successfully in 10 sec, 906 ms (10 minutes ago)

7:15 LF UTF-8 4 spaces

Let's do the same to the class OracleConnection

```
1 /**
2  * @author Rodrigo Martins Pagliares
3  */
```

```
5 public class ReservationService {
6     private OracleConnection connection;
```

```
8     public ReservationService() { connection = new OracleConnection(); }
```

```
12    public void createReservation() {
13        connection.connect();
```

```
14        System.out.println("business logic over the creation of a reservation");
```

Notice that ReservationService, RoomService (not shown here), and ReservationReport (not shown here) are all **coupled** to the OracleConnection **implementation** (our **implementation** in the second version of the exercise). This is what we call **program to an implementation** (OracleConnection) and **not** to an **interface**. Let's change these **3 classes** to use the **design principle** *program to an interface (or abstract class) and not to an implementation*.

End > src > main > java > ReservationReport

Project

- End [Program to an Interface] ~/l
 - .idea
 - src
 - main
 - java
 - Connection
 - MysqlConnection
 - OracleConnection
 - ReservationReport
 - ReservationService
 - RoomService
 - TestConnection
 - resources
 - test
 - target
 - pom.xml
 - External Libraries
 - Scratches and Consoles

Connection.java x MysqlConnection.java x OracleConnection.java x ReservationService.java x ReservationReport.java x

```
1  /**
2    * @author Rodrigo Martins Pagliares
3    */
4
5    public class ReservationReport {
6        private Connection connection;
7
8    public ReservationReport() {
9        connection = new OracleConnection();
10    }
11
12    public void reportGeneration() {
13        connection.connect();
14        System.out.println("business logic in order to generate a report");
15    }
16 }
17
```

Let's

Update ReservationReport, RoomService (not shown here), and ReservationService (not shown here) **program** to an **interface** and **not** to an **implementation**.

End > src > main > java > ReservationReport > ReservationReport

Project

- End [Program to an Interface] ~/l
 - .idea
 - src
 - main
 - java
 - Connection
 - MysqlConnection
 - OracleConnection
 - ReservationReport
 - ReservationService
 - RoomService
 - TestConnection
 - resources
 - test
 - target
 - pom.xml
 - External Libraries
 - Scratches and Consoles

Connection.java x MysqlConnection.java x OracleConnection.java x ReservationService.java x ReservationReport.java x

```
1  /**
2      * @author Rodrigo Martins Pagliares
3      */
4
5      public class ReservationReport {
6          private Connection connection;
7
8          public ReservationReport() {
9              connection = new OracleConnection();
10         }
11
12         public void reportGeneration() {
13             connection.connect();
14             System.out.println("business logic in order to generate a report");
15         }
16     }
17
```

Let's

But we still have a problem: there are 3 classes, ReservationReport, RoomService (not shown here), and ReservationService that is coupled to an **implementation** (OracleConnection). See their constructors.

End > src > main > java > ReservationReport

Project

- End [Program to an Interface] ~/l
- .idea
- src
 - main
 - java
 - Connection
 - MysqlConnection
 - OracleConnection
 - ReservationReport
 - ReservationService
 - RoomService
 - TestConnection
 - resources
 - test
 - target
 - pom.xml
 - External Libraries
 - Scratches and Consoles

ReservationReport.java

```
1  /**
2    * @author Rodrigo Martins Pagliares
3    */
4
5    1 related problem
6    public class ReservationReport {
7
8        private Connection connection;
9
10   1 related problem
11   public ReservationReport(Connection connection) {
12       this.connection = connection;
13   }
14
15   public void reportGeneration() {
16       connection.connect();
17   }
18 }
```

To fix that, let's include an argument to the constructors of the 3 classes. The type of the argument is Connection (an interface and not an implementation). We aren't now instantiating the OracleConnection Class inside the constructor. We will pass an instance to the constructor later. For now, ignore the IntelliJ errors (1 related problem). We are gonna fix them soon

End > src > main > java > TestConnection > main

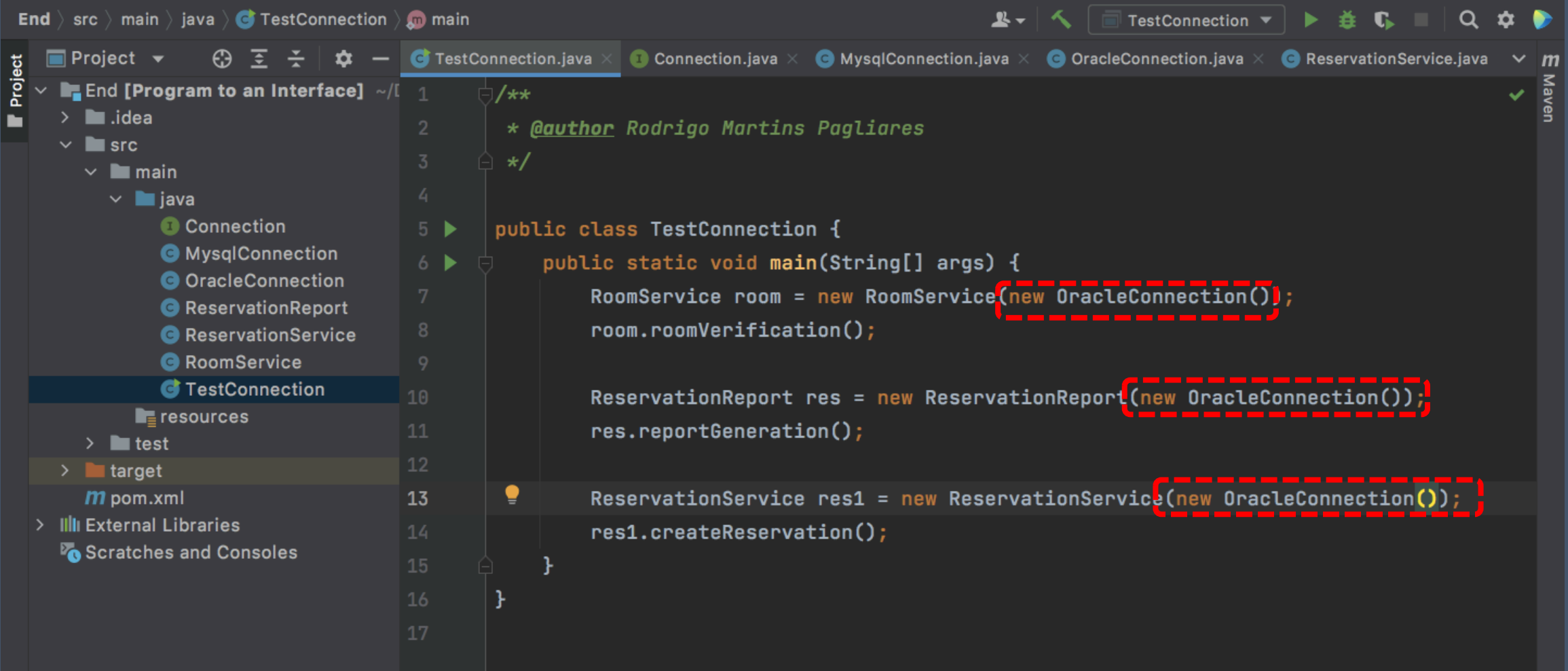
Project

- End [Program to an Interface] ~/l
 - .idea
 - src
 - main
 - java
 - Connection
 - MysqlConnection
 - OracleConnection
 - ReservationReport
 - ReservationService
 - RoomService
 - TestConnection**
 - resources
 - test
 - target
 - pom.xml
 - External Libraries
 - Scratches and Consoles

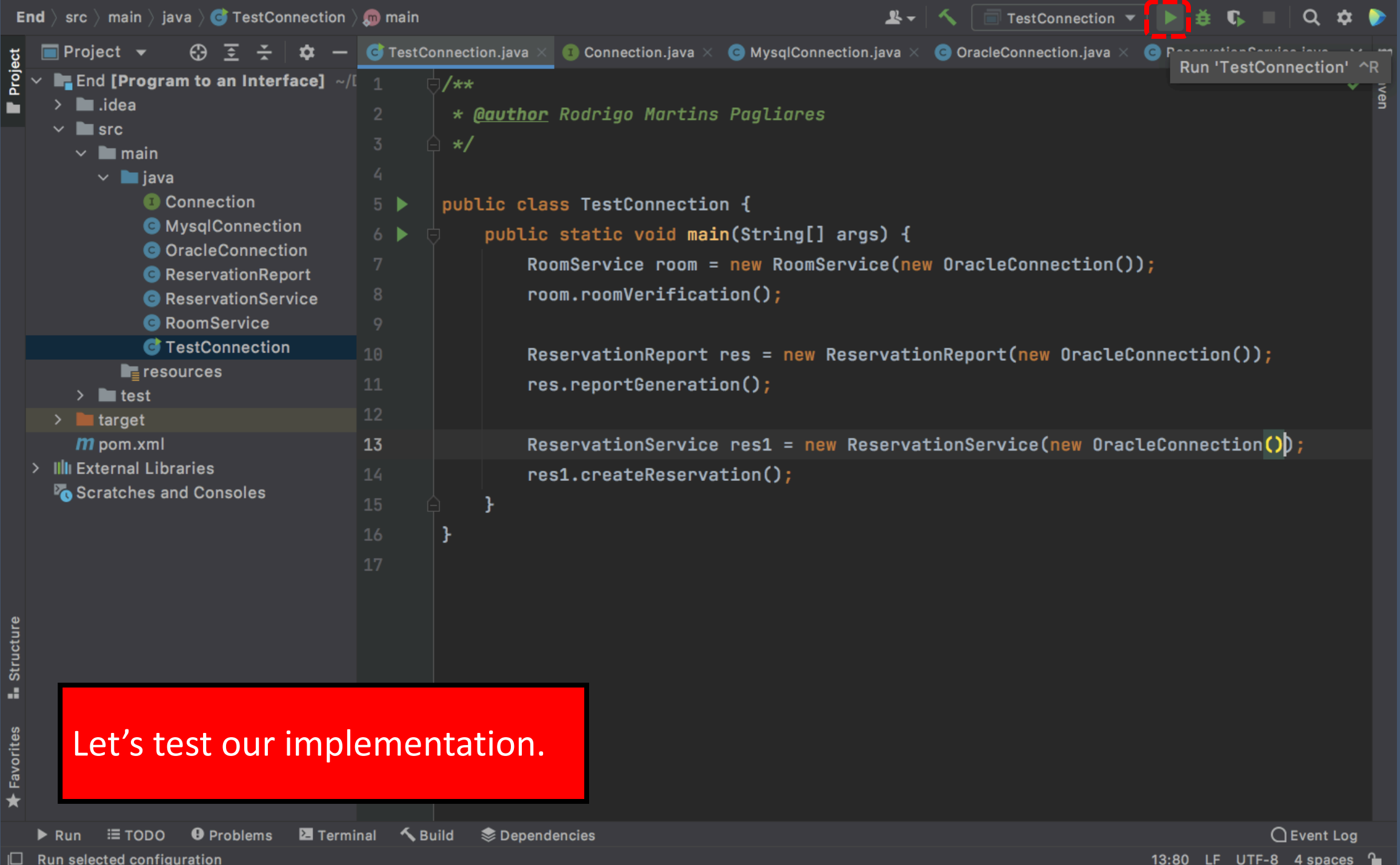
TestConnection.java

```
1  /**
2      * @author Rodrigo Martins Pagliares
3      */
4
5  public class TestConnection {
6      public static void main(String[] args) {
7          RoomService room = new RoomService();
8          room.roomVerification();
9
10         ReservationReport res = new ReservationReport();
11         res.reportGeneration();
12
13         ReservationService res1 = new ReservationService();
14         res1.createReservation();
15     }
16 }
17
```

Let's fix the errors. Notice that we need to update the class with the method main, since now we need to pass an instance of type Connection to the constructors of RoomService, ReservationReport and ReservationService



What about passing an instance of OracleConnection (an implementation of type Connection)?



End > src > main > java > TestConnection > main

Project

- End [Program to an Interface] ~/l
- .idea
- src
 - main
 - java
 - Connection
 - MysqlConnect
 - OracleConne
 - ReservationR
 - ReservationS
 - RoomService
 - TestConnect
 - resources
 - test
 - target
 - pom.xml

TestConnection.java

```
4  
5 public class TestConnection {  
6     public static void main(String[] args) {  
7         RoomService room = new RoomService(new OracleConnection());  
8     }  
9 }
```

Run: TestConnection

```
/Library/Java/JavaVirtualMachines/adoptopenjdk-16.jdk/Contents/Home/bin/java -javaagent:/Applications/IntelliJ IDEA CE  
Connecting to Oracle  
business logic over entity room  
Connecting to Oracle  
business logic in order to generate a report  
Connecting to Oracle  
business logic over the creation of a reservation  
  
Process finished with exit code 0
```

Build completed successfully in 8 sec, 437 ms (moments ago)

13:80 LF UTF-8 4 spaces

You should see the same results of the previous version of our code. However, what is the advantage of updating the 3 classes to program to an interface (Connection) and not to an implementation (OracleConnection), since we got the same results?

```
End > src > main > java > TestConnection > main
Project
  End [Program to an Interface] ~/l
  > .idea
  > src
    > main
      > java
        Connection
        MySqlConnection
        OracleConnection
        ReservationReport
        ReservationService
        RoomService
        TestConnection
      resources
      > test
      > target
      pom.xml
  Run: TestConnection x
  Maven
```

```
4
5 public class TestConnection {
6     public static void main(String[] args) {
7         RoomService room = new RoomService(new OracleConnection());
8         room.roomVerification();
9
10        ReservationReport res = new ReservationReport(new OracleConnection());
11        res.reportGeneration();
12
13        ReservationService res1 = new ReservationService(new OracleConnection());
14        res1.createReservation();
15    }
16 }
17
```

Advantage #1: To answer, remember that when we update our first version of our project from MySqlConnection to OracleConnection we had to change 3 classes (RoomService, ReservationService, and ReservationReport). Now, if we want to change back to MySQL connection, we just need to change this class with method main. Let's do it (next slide).

End > src > main > java > TestConnection > main

Project

- End [Program to an Interface] ~/l
 - .idea
 - src
 - main
 - java
 - Connection
 - MysqlConnection
 - OracleConnection
 - ReservationReport
 - ReservationService
 - RoomService
 - TestConnection**
 - resources
 - test
 - target
 - pom.xml

TestConnection.java

```
4
5 public class TestConnection {
6     public static void main(String[] args) {
7         RoomService room = new RoomService(new MysqlConnection());
8         room.roomVerification();
9
10        ReservationReport res = new ReservationReport(new MysqlConnection());
11        res.reportGeneration();
12
13        ReservationService res1 = new ReservationService(new MysqlConnection());
14        res1.createReservation();
15    }
16 }
17 }
```

Run: TestConnection

```
/Library/Java/JavaVirtualMachines/adoptopenjdk-16.jdk/Contents/Home/bin/java -javaagent:/Applications/IntelliJ IDEA CE
Connecting to MySQL
business logic over entity room
Connecting to MySQL
business logic in order to generate a report
Connecting to MySQL
business logic over the creation of a reservatio

Process finished with exit code 0
```

Build completed successfully in 3 sec, 563 ms (moments ago)

12:1 LF UTF-8 4 spaces

Advantage #1: We are working with MySQL now !

End > src > main > java > TestConnection > main

Project

- End [Program to an Interface] ~/l
 - .idea
 - src
 - main
 - java
 - Connection
 - MysqlConnection
 - OracleConnection
 - ReservationReport
 - ReservationService
 - RoomService
 - TestConnection**
 - resources
 - test
 - target
 - pom.xml

TestConnection.java

```
4
5 public class TestConnection {
6     public static void main(String[] args) {
7         RoomService room = new RoomService(new MysqlConnection());
8         room.roomVerification();
9
10        ReservationReport res = new ReservationReport(new OracleConnection());
11        res.reportGeneration();
12
13        ReservationService res1 = new ReservationService(new MysqlConnection());
14        res1.createReservation();
15    }
16 }
17 }
```

Run: TestConnection

/Library/Java/JavaVirtualMachines/adoptopenjdk-16.jdk/Contents/Home/bin/java -javaagent:/Applications/IntelliJ IDEA CE

Connecting to MySQL

business

Connecti

business

Connecti

business

Process

Run

TODO

Problems

Terminal

Build

Dependencies

Event Log

Build completed successfully in 4 sec, 44 ms (moments ago)

8:33 LF UTF-8 4 spaces

Advantage #2: Since we are programming to an interface, we can make our code work with ANY implementation of the interface. We can even change the implementations during runtime. Let's try to use MysqlConnection and OracleConnection in the same program.

End > src > main > java > TestConnection > main

Project

- End [Program to an Interface] ~/l
 - .idea
 - src
 - main
 - java
 - Connection
 - MysqlConnection
 - OracleConnection
 - ReservationReport
 - ReservationService
 - RoomService
 - TestConnection**
 - resources
 - test
 - target
 - pom.xml

TestConnection.java

```
4
5 public class TestConnection {
6     public static void main(String[] args) {
7         RoomService room = new RoomService(new MysqlConnection());
8         room.roomVerification();
9
10        ReservationReport res = new ReservationReport(new OracleConnection());
11        res.reportGeneration();
12
13        ReservationService res1 = new ReservationService(new MysqlConnection());
14        res1.createReservation();
15    }
16 }
17 }
```

Run: TestConnection

```
/Library/Java/JavaVirtualMachines/adoptopenjdk-16.jdk/Contents/Home/bin/java -javaagent:/Applications/IntelliJ IDEA CE
Connecting to MySQL
business logic over entity room
Connecting to Oracle
business logic in order to generate a report
Connecting to MySQL
business logic over the creation of a reservatio

Process finished with exit code 0
```

Build completed successfully in 4 sec, 44 ms (2 minutes ago)

8:33 LF UTF-8 4 spaces

Advantage #2: We are now working with MySQL AND Oracle !

End > src > main > java > TestConnection > main

Project

- End [Program to an Interface] ~/l
 - .idea
 - src
 - main
 - java
 - Connection
 - MysqlConnection
 - OracleConnection
 - ReservationReport
 - ReservationService
 - RoomService
 - TestConnection**
 - resources
 - test
 - target
 - pom.xml

TestConnection.java

```
4
5 public class TestConnection {
6     public static void main(String[] args) {
7         RoomService room = new RoomService(new MysqlConnection());
8         room.roomVerification();
9
10        ReservationReport res = new ReservationReport(new OracleConnection());
11        res.reportGeneration();
12
13        ReservationService res1 = new ReservationService(new MysqlConnection());
14        res1.createReservation();
15    }
16 }
17 }
```

Run: TestConnection

```
/Library/Java/JavaVirtualMachines/adoptopenjdk-16.jdk/Contents/Home/bin/java -javaagent:/Applications/IntelliJ IDEA CE
Connecting to MySQL
business logic over entity room
Connecting to Oracle
business logic in order to generat
Connecting to MySQL
business logic over the creation c

Process finished with exit code 0
```

Advantage #3: What do we need to do to support another RDBMS, for example, SQL Server?

Run | TODO | Problems | Terminal | Build | Dependencies | Event Log

Build completed successfully in 4 sec, 44 ms (2 minutes ago) 8:33 LF UTF-8 4 spaces

End > src > main > java > **SQLServerConnection**

Project

- End [Program to an Interface] ~/l
 - .idea
 - src
 - main
 - java
 - Connection
 - MySQLConnection
 - OracleConnection
 - ReservationReport
 - ReservationService
 - RoomService
 - SQLServerConnection**
 - TestConnection
 - resources
 - test
 - target
 - pom.xml
 - External Libraries
 - Scratches and Consoles

Structure

Run: TestConnection

Run

Build completed successfully in 7 sec, 351 ms (9 minutes ago)

```
1  /**
2      * @author Rodrigo Martins Pagliares
3      */
4
5  public class SQLServerConnection implements Connection {
6
7      @Override
8      public void connect() {
9          System.out.println("Connection to SQLServer");
10     }
11 }
12
```

Advantage #3: First we need to create the SQLServerConnection class that implements the Connection interface

End > src > main > java > ReservationService > createReservation

Project

- End [Program to an Interface] ~/l
 - .idea
 - src
 - main
 - java
 - Connection
 - MysqlConnection
 - OracleConnection
 - ReservationReport
 - ReservationService
 - RoomService
 - SQLServerConnection
 - TestConnection
 - resources
 - test
 - target
 - pom.xml
 - External Libraries
 - Scratches and Consoles

stConnection.java x SQLServerConnection.java x Connection.java x OracleConnection.java x ReservationService.java x

```
1  /**
2      * @author Rodrigo Martins Pagliares
3      */
4
5      public class ReservationService {
6          private Connection connection;
7
8          public ReservationService(Connection connection) {
9              this.connection = connection;
10         }
11
12         public void createReservation() {
13             connection.connect();
14             System.out.println("business logic over the creation of a reservation");
15         }
16     }
```

Advantage #3: Since ReservationService, RoomService and ReservationReport are coupled with the interface (Connection) and not with a specific implementation (such as OracleConnection, SQLConnection, etc), no changes are needed in these 3 classes (Contrast with the first version when we had to change the code of these 3 classes to change from MySQLConnection to OracleConnection)

End > src > main > java > TestConnection > main

Project

- End [Program to an Interface] ~/l
 - .idea
 - src
 - main
 - java
 - Connection
 - MysqlConnection
 - OracleConnection
 - ReservationReport
 - ReservationService
 - RoomService
 - SQLServerConnection
 - TestConnection**
 - resources
 - test
 - target
 - pom.xml
 - External Libraries
 - Scratches and Consoles

TestConnection.java

```
1  /**
2      * @author Rodrigo Martins Pagliares
3      */
4
5  public class TestConnection {
6      public static void main(String[] args) {
7          RoomService room = new RoomService(new MysqlConnection());
8          room.roomVerification();
9
10         ReservationReport res = new ReservationReport(new OracleConnection());
11         res.reportGeneration();
12
13         ReservationService res1 = new ReservationService(new MysqlConnection());
14         res1.createReservation();
15     }
16 }
17
```

Run: TestConnection

Structure

Run

Build completed successfully in 4 sec, 44 ms (today 17:01)

8:33 LF UTF-8 4 spaces

Advantage #3: Instead of changing 3 classes, we just need to include support to our newly created SQLServerConnection in this class with the method main. Let's do it (next slide)

End > src > main > java > TestConnection > main

Project

- End [Program to an Interface] ~/l
- > .idea
- > src
 - main
 - java
 - Connection
 - MysqlConnection
 - OracleConnection
 - ReservationReport
 - ReservationService
 - RoomService
 - SqlServerConnection
 - TestConnection**
 - resources
 - test
 - target

TestConnection.java

```
4
5 public class TestConnection {
6     public static void main(String[] args) {
7         RoomService room = new RoomService(new MysqlConnection());
8         room.roomVerification();
9
10        ReservationReport res = new ReservationReport(new OracleConnection());
11        res.reportGeneration();
12
13        ReservationService res1 = new ReservationService(new SqlServerConnection());
14        res1.createReservation();
15    }
16 }
```

Run: TestConnection

/Library/Java/JavaVirtualMachines/adoptopenjdk-16.j

- Connecting to MySQL
- business logic over entity room
- Connecting to Oracle
- business logic in order to generate a report
- Connection to SQLServer
- business logic over the creation of a reservation

Process finished with exit code 0

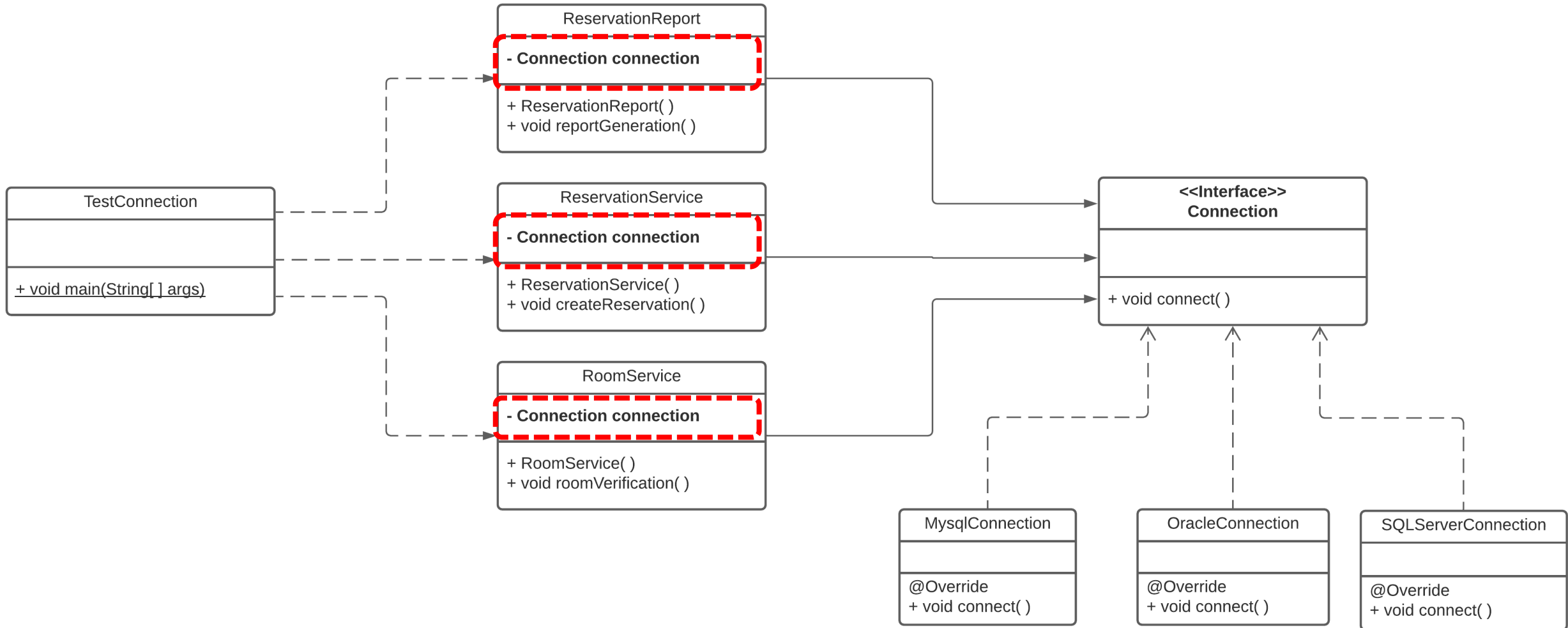
Run | TODO | Problems | Terminal | Build | Dependencies

Build completed successfully in 7 sec, 351 ms (moments ago)

10:1 LF UTF-8 4 spaces

Advantage #3: Included support to SQL Server. Now our code works with MySQL, Oracle and SQL server. By using the design principle of program to an interface we can extend our design to include new types of connections without need to change the classes ReservationService, RoomService and ReservationReport.

Class diagram: version #3 (final)



Questions?